MODULE 3 · MACHINE LEARNING · DAY 35

Yes or No

Logistic regression

The same line as yesterday, bent into a probability.

5	16	1
sections	worked pictures	extra step



Boston University

Prepared & presented by **Srinivasa Sai Chava**

WHERE WE ARE

Until today, the answer was always a number

1 Day 33

One column, one line. Errors, least squares, R^2.

2 Day 34

Many columns. One coefficient each.

3 Yesterday

The cost function, and gradient descent.

4 TODAY

The answer is a category. Pass or fail.

You already have every piece of this

The line is Day 34's, unchanged. The idea of a cost function and of walking downhill is yesterday's, unchanged. Today adds exactly two things: a way to turn any number into a probability, and a cost function that is honest about confident mistakes.

Despite the name, logistic regression is a CLASSIFIER. It is the single most used model in industry, and it is the one you should try before anything clever.

SECTION 1 OF 5

Why a line will not do

The answer is 0 or 1. A line has never heard of that.

- Ten students
- A line keeps going
- What we need instead

THE PROBLEM

Ten students. This time the answer is pass or fail.

EXAMPLE **Same question as Day 33 — but instead of a mark out of 100, we only record whether they passed.**

hours	1	2	3	4	5	6	7	8	9	10
passed?	no	no	no	no	YES	no	YES	YES	YES	YES

Note the two awkward ones: somebody passed on 5 hours, somebody failed on 6. Real data always looks like this.

Day 33

predict a MARK — a number

→

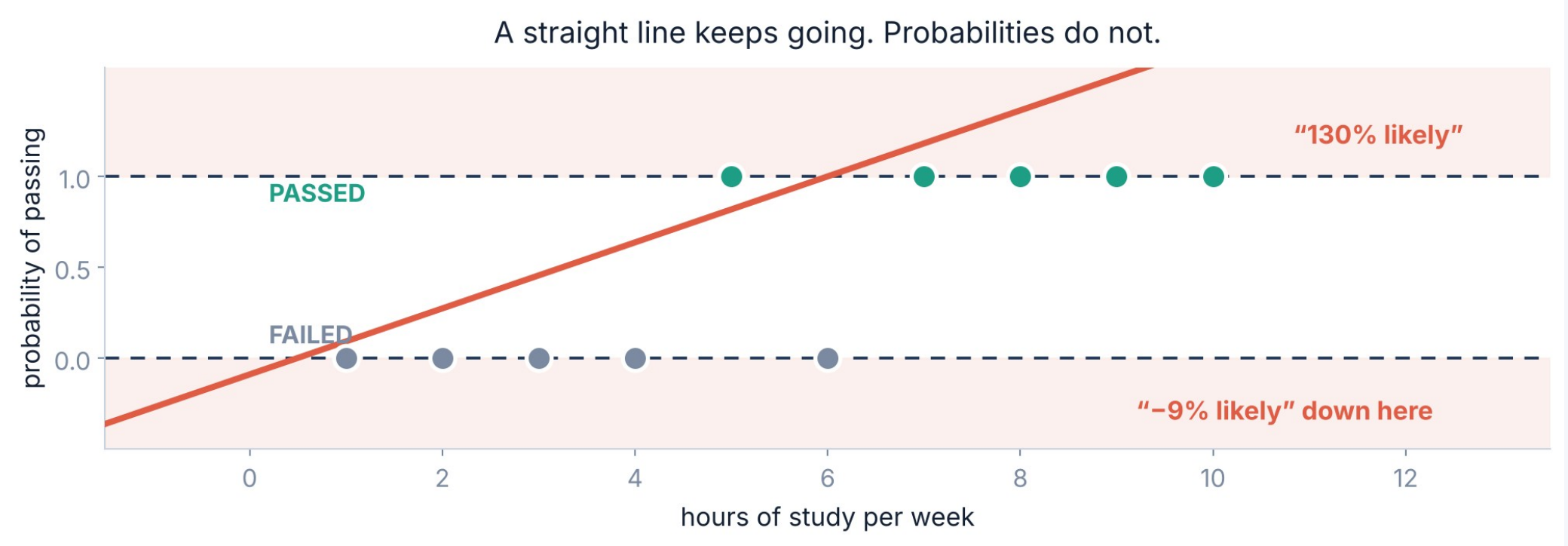
Today

predict PASS or FAIL — a category

Most real questions are this shape. Will this customer leave? Is this transaction fraud? Does this scan show a tumour? Yes or no, not how much.

THE OBVIOUS ATTEMPT

Put a straight line through it and watch what happens



A line is a ramp that never stops. Probabilities live strictly between 0 and 1. Those two facts cannot both hold — so the line has to be bent.

THE REQUIREMENT

So what do we actually want the model to give us?

A NUMBER BETWEEN 0 AND 1

Not a yes or a no. A probability — “this student has a 0.72 chance of passing”. Much more useful than a bare verdict.

THAT NEVER ESCAPES

However extreme the input, the answer must stay inside 0 to 1. At 100 hours the answer should approach 1, not sail past it.

AND STILL BUILT ON A LINE

We want to keep the coefficients, the interpretability, the speed — everything good about Day 34.

Keep the line. Then run its output through one function that squashes any number — however huge, however negative — into the range 0 to 1. That function is the whole trick.

SECTION 2 OF 5

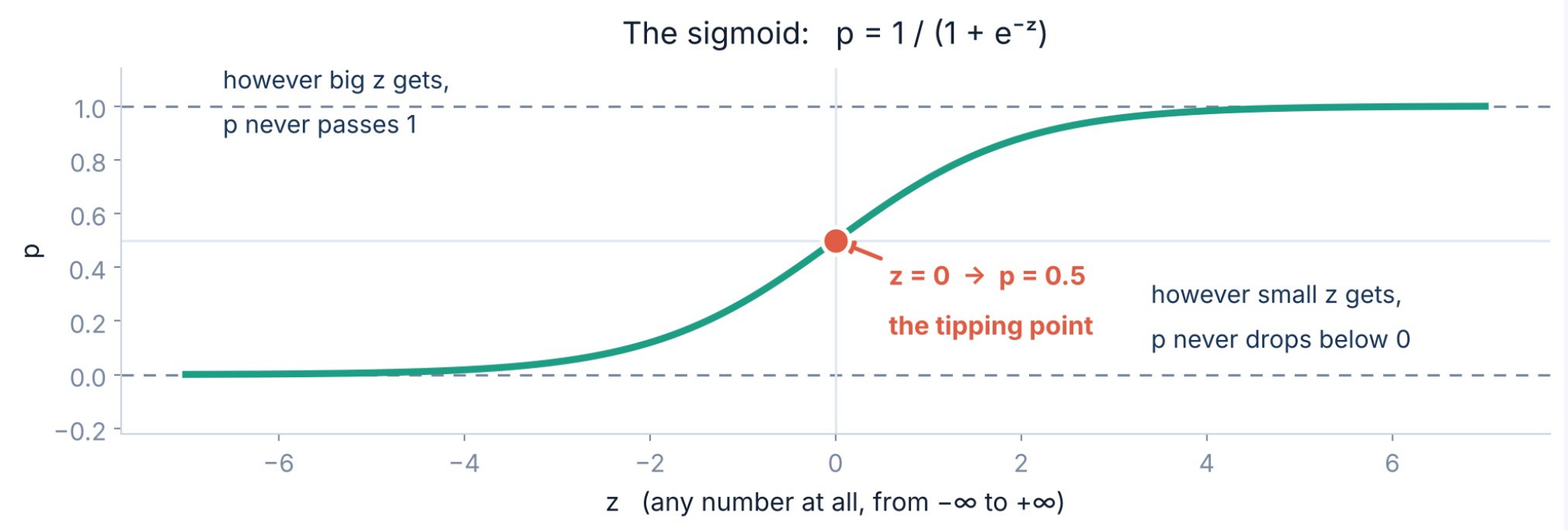
Squashing it into 0 to 1

One function does the whole job.

- The sigmoid
- The two steps
- The fitted curve
- Reading probabilities

THE ONE NEW FUNCTION

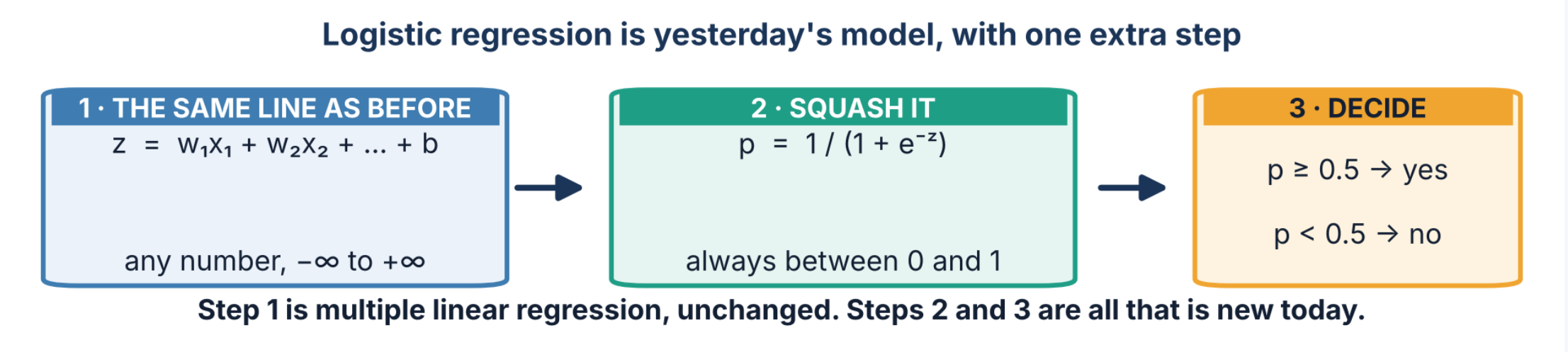
The sigmoid



Feed it −1,000 and you get 0.0000. Feed it +1,000 and you get 1.0000. Feed it 0 and you get exactly 0.5. It can never leave the range, no matter what you give it.

HOW IT FITS TOGETHER

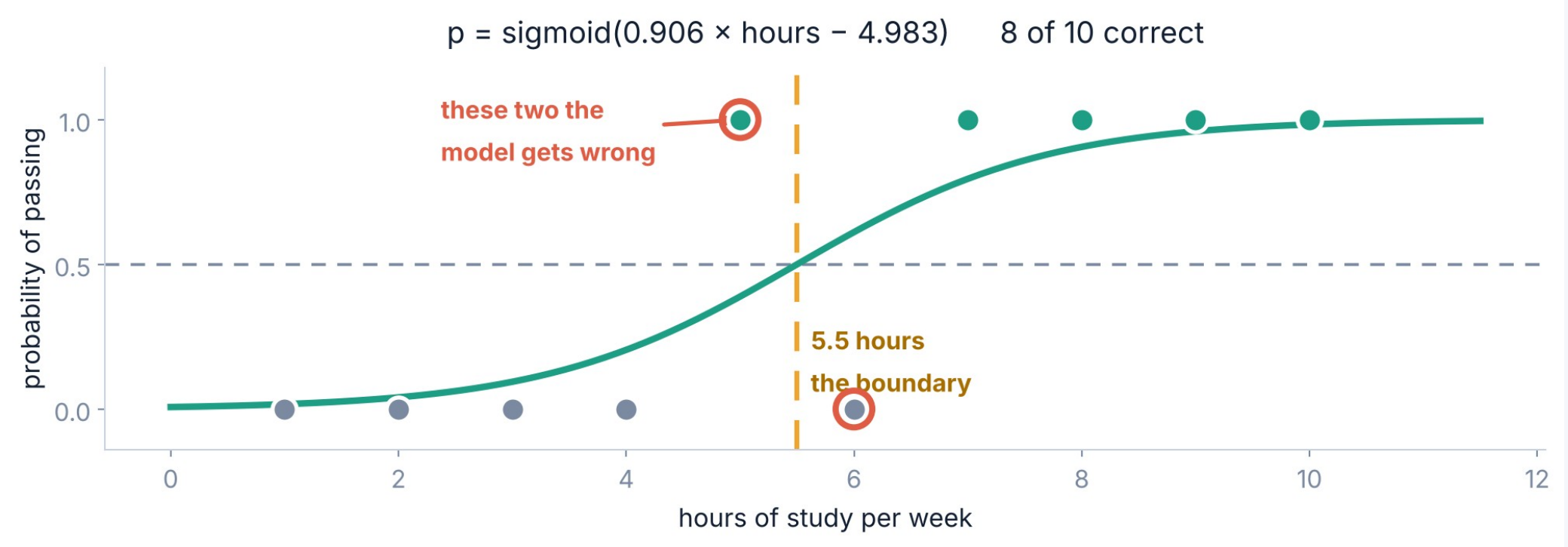
The whole model, in two steps



An example. A student studies 7 hours. Step 1 gives $z = 0.906 \times 7 - 4.983 = 1.359$. Step 2 gives $p = 1 / (1 + e^{-1.359}) = 0.796$. Step 3: 0.796 is above 0.5, so we predict a pass — and we can honestly say “about a 4-in-5 chance”.

THE RESULT

Fitted to our ten students



The curve crosses 0.5 at exactly 5.5 hours. Below that the model says fail, above it says pass — and it gets 8 of the 10 right, missing precisely the two awkward students.

READING THE OUTPUT

The probability for every student

hours	z = 0.906 × hours − 4.983	p = sigmoid(z)	we predict	truth
1	−4.077	0.017	fail	fail
3	−2.265	0.094	fail	fail
5	−0.453	0.389	fail	PASS ×
6	+0.453	0.611	pass	fail ×
7	+1.359	0.796	pass	pass
10	+4.076	0.983	pass	pass

The two it gets wrong are the two closest to the boundary — 0.389 and 0.611. The model was not confident about either, and it was right not to be.

This is why a probability beats a verdict. “Fail” and “fail, probability 0.389” are very different things to tell a student in week three.

SECTION 3 OF 5

A cost that works

Yesterday's machinery, with the right cost function.

- Why squared error fails
- Log loss
- Gradient descent, unchanged

PICKING UP WHERE YOU LEFT OFF

Everything from yesterday carries straight over

YOU ALREADY HAVE

A cost function is a single number saying how wrong the model is.

Gradient descent finds the numbers that make it smallest: guess, measure, step downhill, repeat.

TODAY NEEDS ONE CHANGE

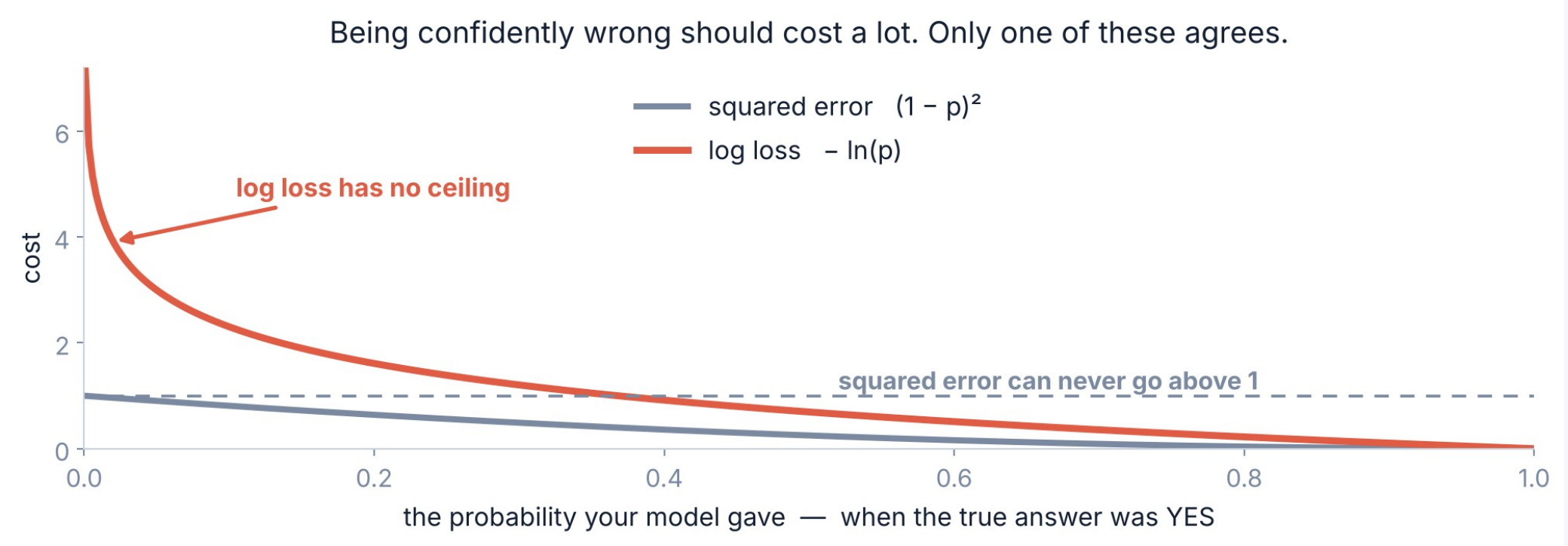
The cost function itself.

Squared error was right for predicting a mark. For predicting a yes or a no it is the wrong tool — and the next two slides show exactly why.

Gradient descent does not change at all today. Same loop, same learning rate, same “walk downhill”. Only what it is walking down is different.

THE PROBLEM

Why squared error is the wrong cost for a yes/no answer



A student really did pass. Your model said 0.001. Squared error charges you 0.998 — barely more than if you had said 0.1. Log loss charges you 6.91.

ONE DISASTER IN A HUNDRED

What that ceiling costs you in practice

100 students. The model is right about 99 of them, and catastrophically wrong about one.

SQUARED ERROR	LOG LOSS
average cost 0.0199	average cost 0.1734
the disaster barely registers	**the disaster is 40% of the total**

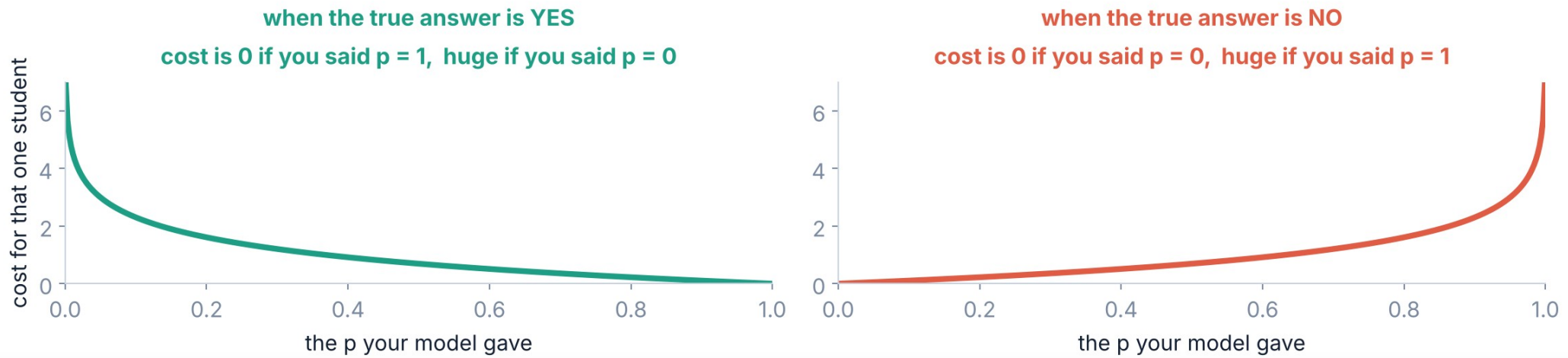
That one student was a real person the model told to relax. The cost function has to notice.

Be honest about what this does and does not prove. On clean data, fitting by squared error gives almost the same curve as log loss. The argument is not that the fit falls apart — it is that squared error cannot express how bad a confident mistake is, and in medicine, credit and fraud that is exactly the mistake that matters.

THE FIX

Log loss: the cost that has no ceiling

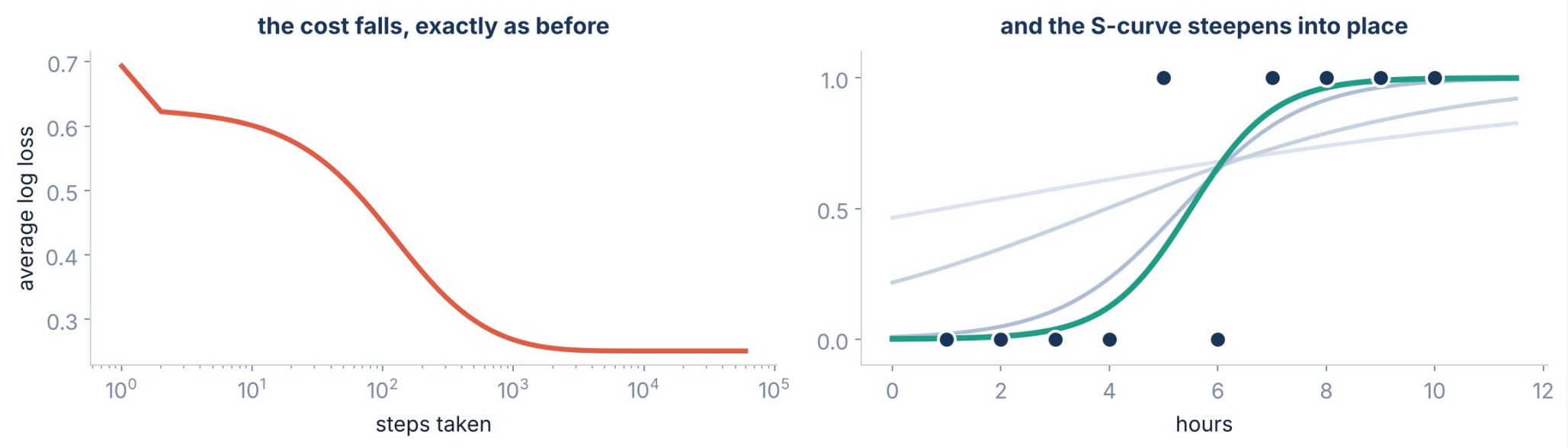
log loss = − ln(p) if the answer was yes, − ln(1 − p) if it was no



Every student contributes one of the two. Add them all up, divide by how many — that is the number gradient descent walks down.

FITTING THE MODEL

Gradient descent, exactly as you learned it



Same loop as yesterday. There is no formula for logistic regression — no shortcut like the one Day 33 had — so walking downhill is not one option among two. It is the only way.

WORTH KNOWING NOW

One honest note about what scikit-learn gives you

how it was fitted	coefficient	intercept	boundary
our own gradient descent, run to the end	1.302	−7.159	5.50 hours
`LogisticRegression(C=1e9)`	1.302	−7.159	5.50 hours
`LogisticRegression()` ← the default	**0.906**	**−4.983**	5.50 hours

The default is not the pure answer. scikit-learn applies a little REGULARIZATION unless you tell it not to — it deliberately shrinks the coefficients to stop the model becoming over-confident. That is why 1.302 becomes 0.906. The boundary is identical either way, and on real data the shrunk version usually generalises better. We give it a full day on Day 48.

Flagging it now so that when you compute a coefficient by hand and scikit-learn disagrees, you know why — and you know the one argument that makes them agree.

SECTION 4 OF 5

Reading the answer

A coefficient here does not mean what you think.

- Not probability
- Odds
- The boundary
- The cut-off

THE TRAP

The coefficient is 0.906. It is NOT “+0.906 probability”.

WHAT IT IS NOT

“One extra hour adds 0.906 to the probability.”

Impossible — probabilities stop at 1. Two extra hours would take you past certainty.

WHAT IT IS NOT, EITHER

“One extra hour adds 90.6 percentage points.”

Look at the real numbers: from 4 to 5 hours, p rises by 0.185. From 7 to 8, by 0.110. Different every time.

WHAT IT ACTUALLY IS

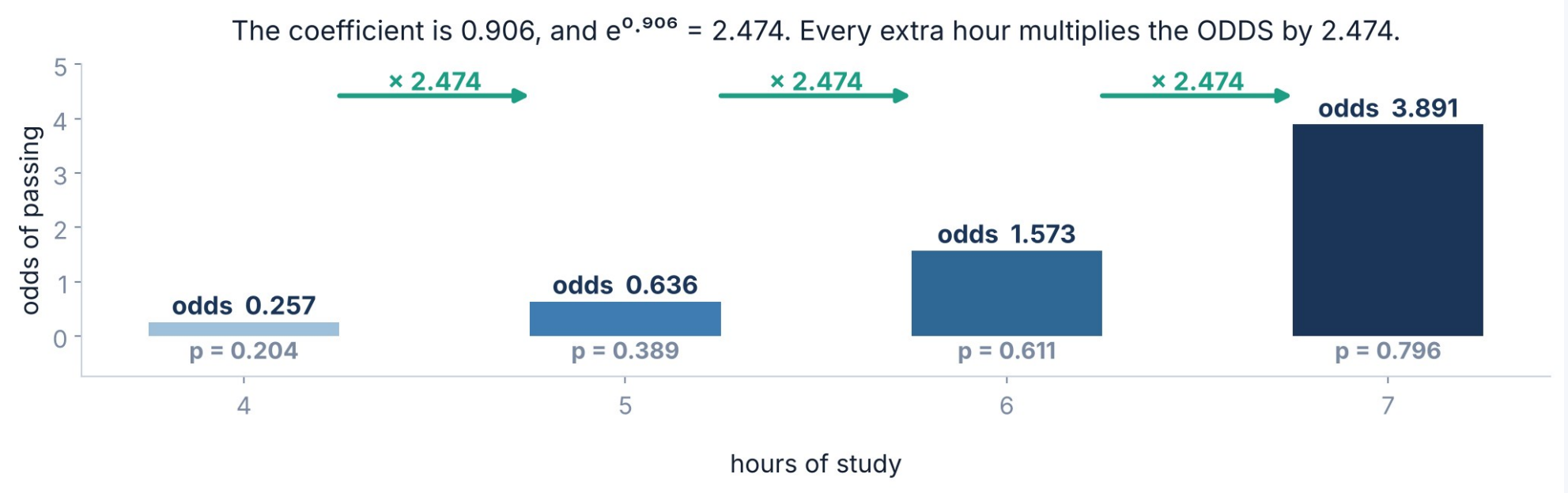
“One extra hour adds 0.906 to the LOG-ODDS.”

Which sounds like jargon until you undo the log — and then it becomes the cleanest sentence in the whole model.

Odds = p ÷ (1 − p). A probability of 0.8 is odds of 4 — “four times more likely than not”. It is the language bookmakers and doctors already use.

WHAT IT MEANS

Undo the log and it reads cleanly



$e^{0.906} = 2.474$. Every extra hour of study multiplies your ODDS of passing by 2.474 — the same multiplier at every point on the curve, which is what makes it worth saying.

THE REASON

Why odds, and not just probability?

Why we talk about odds and not probability

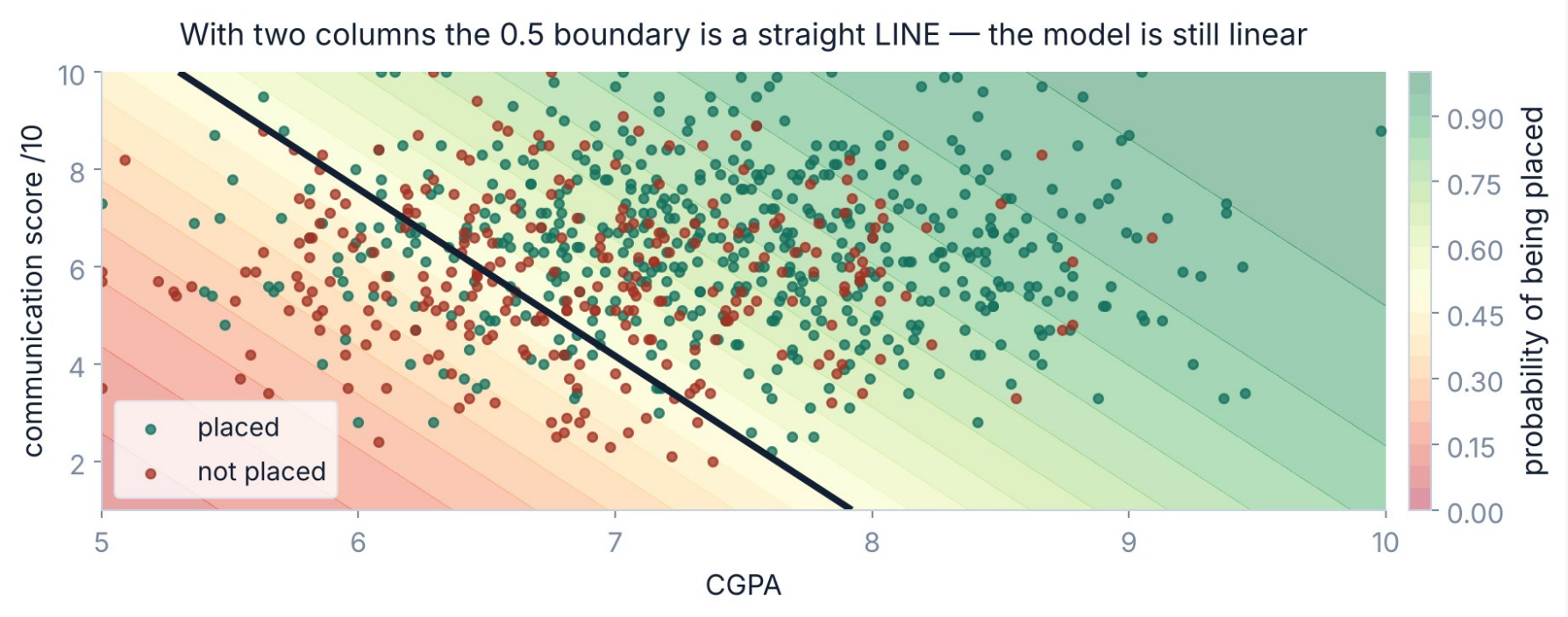
hours	4	5	6	7	8
probability	0.204	0.389	0.611	0.796	0.906
odds	**0.257**	**0.636**	**1.573**	**3.891**	**9.627**

The probability steps are +0.185, +0.222, +0.185, +0.110 — all different. The odds steps are ×2.474 every single time.

The model is linear in the log-odds, not in the probability. That is the whole reason logistic regression keeps the clean, Day-34-style interpretation. A single multiplier describes the effect everywhere, instead of a different number at every point on the curve.

THE DECISION BOUNDARY

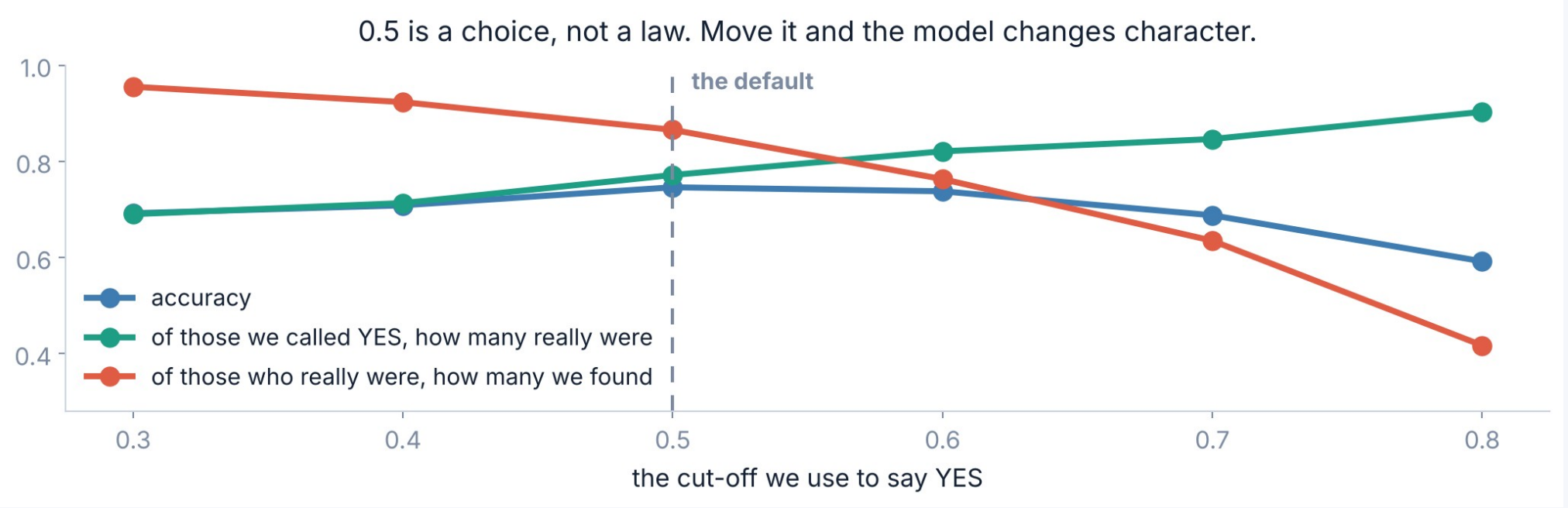
With two columns, the boundary is still a straight line



The CURVE is in the probability. The BOUNDARY is dead straight — which is both why the model is easy to explain, and the first real limit on what it can learn.

THE CUT-OFF

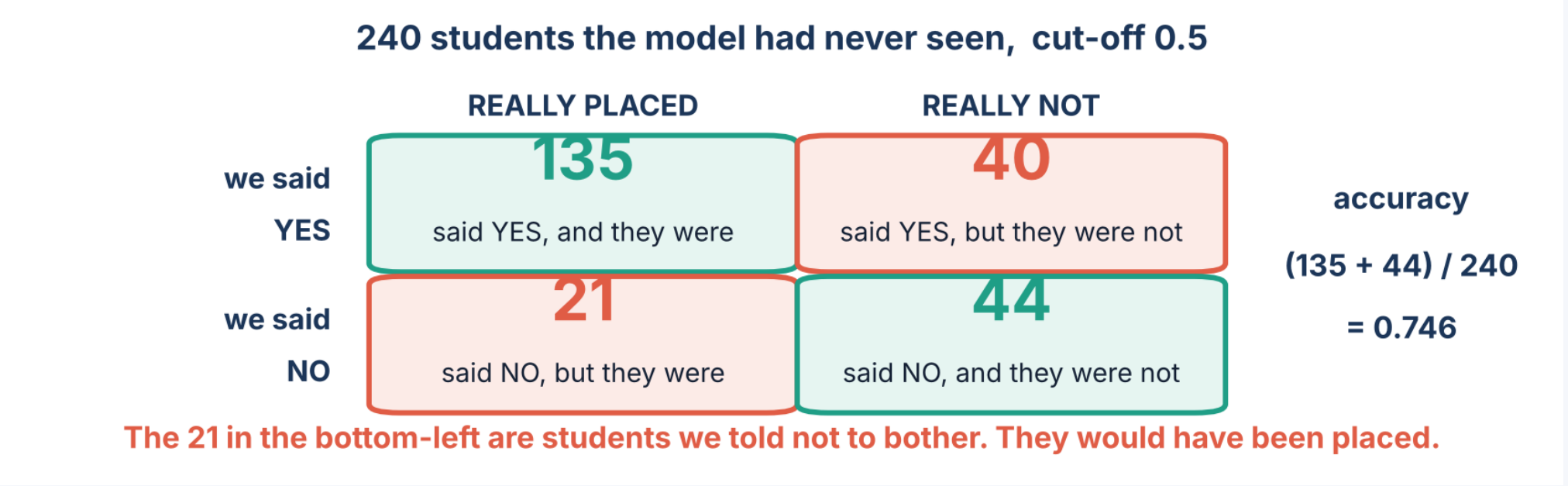
0.5 is a choice, not a law



Raise the cut-off and you are right more often when you say yes — but you miss more of the people who deserved a yes. Nothing about the MODEL changed. You changed what you do with it.

COUNTING THE MISTAKES

The four ways a yes/no model can be right or wrong



Accuracy 0.746 collapses all four of those boxes into one number — and the two kinds of mistake are not the same mistake. Day 47 is entirely about this table.

SECTION 5 OF 5

Doing it for real

Eight hundred students, five columns, and the usual traps.

- The code
- A real model
- Comparing columns
- Beating the baseline

SCIKIT-LEARN

The code is one word different from yesterday

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression().fit(X, y)          #  y is 0 / 1, not a number
model.predict(X_new)                            #  the yes/no verdict
model.predict_proba(X_new)[:, 1]                #  the probability  <- use this one
```

.predict()

gives 0 or 1, using a 0.5 cut-off you never chose

.predict_proba()

gives two columns: P(no) and P(yes). Take column 1

.coef_

log-odds per unit. Run it through exp() before quoting it

.score()

accuracy, not R^2. Always compare it with the baseline

Reach for .predict_proba(), not .predict(). The probability is the model's real output; the verdict is that probability with somebody's threshold stapled on.

A REAL-SIZED MODEL

800 students: will they be placed?

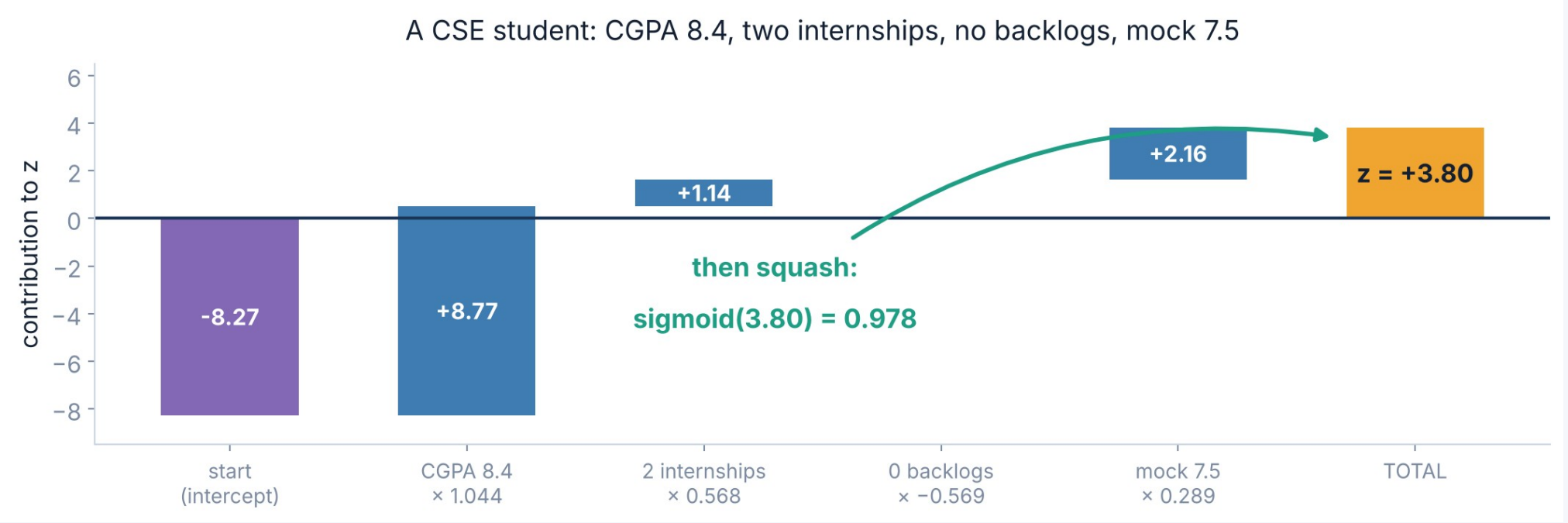
EXAMPLE **CGPA · internships · backlogs · a mock-interview score out of 10 · and their branch.**

column	coefficient	multiply the odds by	in English
`cgpa`	+1.044	× 2.84	one more CGPA point nearly triples the odds
`internships`	+0.568	× 1.76	each internship raises the odds by 76%
`backlogs`	−0.569	× 0.57	each backlog cuts the odds almost in half
`mock_score`	+0.289	× 1.33	each point of communication adds a third
`branch_ECE`	−0.712	× 0.49	about half the odds of a CSE student (the baseline)

Every one of those sentences carries the clause from Day 34: holding the other four columns constant. “Backlogs halve your odds” is only true for two students who match on everything else.

USING THE MODEL

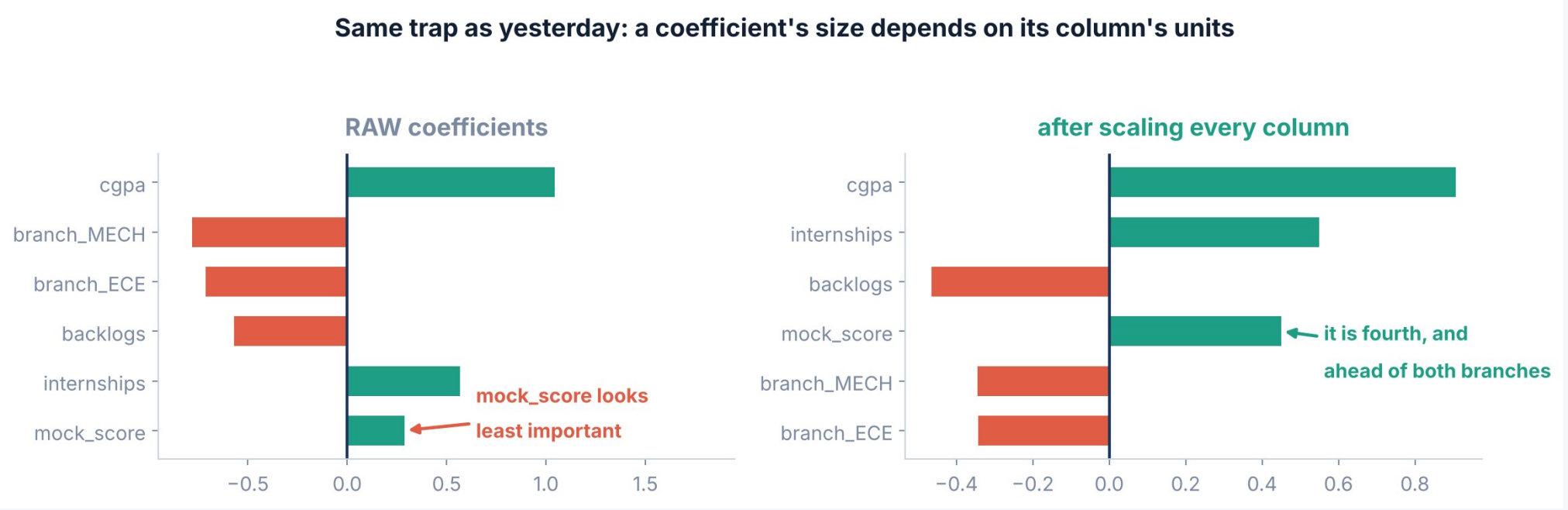
One student, start to finish



Multiply, add, squash. z = +3.80, so p = 0.978 — this student has about a 98% chance. Five multiplications and one exponential, and you can check every step by hand.

COMPARING COLUMNS

Which column matters most? Same trap as yesterday.



Raw, mock_score looks like the least important thing in the model. Scaled, it is fourth and ahead of both branch columns — because a “point” of CGPA and a “point” of mock score are nothing like the same size.

ALWAYS ASK WHAT THE BASELINE IS

Is 74.6% accuracy any good?

65.0%	→	74.6%	→	+9.6
just say “placed” to everybody		our model		percentage points earned

That gap is the only thing that means anything. 65% of these students were placed, so a model that blindly says “yes” to everyone is already right about two-thirds of the time. Quote 74.6% on its own and you sound impressive; quote it next to 65% and you are being honest.

⚠️ And this gets much worse with rare events. If 1 in 100 transactions is fraud, a model that says “never fraud” scores 99% — and is completely useless. That is the Imbalanced Data day.

SPLITTING

One new habit when the answer is a category

```
train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
#                                                     ^^^^^^^^^^
```

WITHOUT stratify

The split is random. With a rare class you can end up with very few — or even none — of the YES rows in one half, and the scores become meaningless.

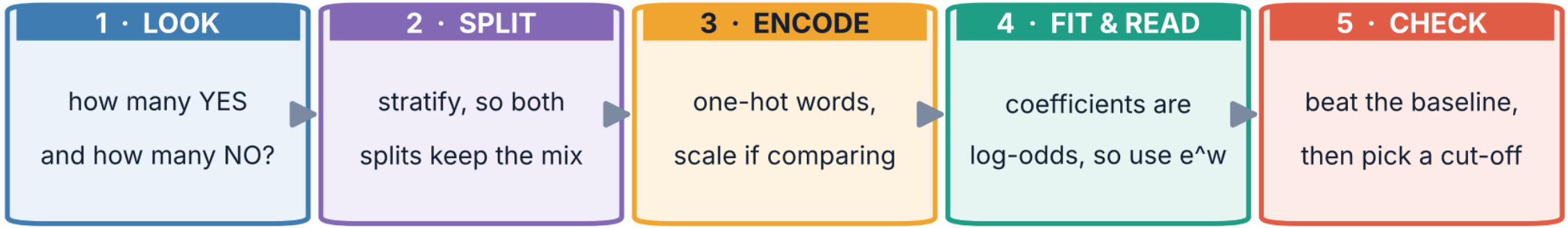
WITH stratify=y

Both halves keep the same mix as the full dataset. Here, 65% placed in the training rows and 65% in the hidden ones.

One word, and it costs nothing. Make it a reflex for every classification problem from today onwards.

PUT IT TOGETHER

The five steps, for a yes/no model



Step 1 changed. For a number you drew a scatter plot. For a category, the first thing you do is count: how many YES and how many NO? That single number tells you the baseline you have to beat, and whether the classes are badly imbalanced.

BEFORE YOU GO

Five mistakes everybody makes once

#	the mistake	what you will see	the fix
1	Reading a coefficient as probability	“One hour adds 0.906 to the chance”	Run it through exp() — it is odds
2	Using .predict() and never the probability	A verdict with no idea how confident	Use .predict_proba()[:, 1]
3	Quoting accuracy with no baseline	“74.6%!” when guessing gives 65%	Always report both numbers
4	Treating 0.5 as fixed	Missing the cases that actually matter	Choose the cut-off from the cost of each mistake
5	Forgetting stratify=y	Wild scores on a rare class	Add it. It costs one word

Number 1 is the one that ends up in a report, and number 3 is the one that ends up in a presentation. Both are easy to avoid and neither is forgiven.

SUMMARY

What to remember from today

A line, then a squash	$z = w_1x_1 + \ldots + b$, exactly as yesterday. Then $p = 1/(1 + e^{-z})$, which can never leave 0 to 1.
Squared error has a ceiling	However confidently wrong you are, it can never charge more than 1. Log loss charges 6.91 for a p of 0.001.
Gradient descent, unchanged	Same loop as yesterday — and this time there is no formula to fall back on, so it is the only route.
Coefficients are log-odds	0.906 is not probability. $e^{0.906} = 2.474$, so one extra hour multiplies the ODDS by 2.474, everywhere on the curve.
0.5 is your choice	The model outputs a probability. The threshold is a business decision, and it belongs to you, not to sklearn.
Accuracy needs a baseline	74.6% against 65% for guessing. The gap is the finding; the number on its own is decoration.

19 QUESTIONS

Tonight's practice

A **Work it out** — 8 questions

Sigmoid by hand, odds, and reading coefficients correctly

B **Find the mistake** — 4 questions

Four programs: a misread coefficient, a missing baseline, a bad cut-off

C **Write the code** — 5 tasks

Fit, read in odds, pick a threshold and defend it

D **Challenge** — 2 tasks

Build logistic regression from scratch, then judge a model that screens job applicants

Part D asks you to write the gradient descent loop yourself — the one from yesterday, with the sigmoid in it — and confirm it lands where scikit-learn lands.

$$p = 1 / (1 + e^{-z})$$

One function, wrapped around the line you already knew how to build. Every classifier for the rest of this course is a different answer to the same question: where do I draw the boundary?

NEXT · DAY 36

Train/test split and cross-validation — how to measure any of this honestly.

Slides & notebooks by **Srinivasa Sai Chava** · Boston University